AMERICAN LATIN CLASS ATTENDANCE SYSTEM

README

Table of Contents

Postman Manual Verification

Import these two files into Postman:

- `American-Latin-Class.postman_environment.json`
- `American-Latin-Class-API.postman_collection.json`
- `American-Latin-Class-Analytics-API.postman_collection.json`

The environment now includes real RDS verification IDs from seed `REAL-20260624154645`; the old dummy fixture IDs were removed.

Manual order:

1. Select the `American Latin Class - AWS` environment.
2. Run `Auth & Session / Auth Config`.
3. Run `Auth & Session / Current Session - No Token`; it must return `401 Authentication required`.
4. Run `Auth & Session / Password Login - Invalid Credentials`; it must return `401`.
5. Run `Auth & Session / Password Login - Demo`. Postman sends `login_email` and `login_password`, then stores `data.sessionToken` in `session_token`.
6. Open any protected request, choose Authorization type `Bearer Token`, and use `{{session_token}}`. The collection already does this automatically at collection level.
7. Run `Auth & Session / Current Session - Bearer Token`; it must return `200` and the logged-in user.
8. Run the remaining folders in order: Public Enrollment, Identity And RBAC, Catalog CRUD, Attendance And Absences, Reports And Evaluations.
9. Run `Session Teardown / Logout` and then `Session Teardown / Current Session - After Logout`; the same token must return `401`.

Python Analytics API order:

1. Run `Auth & Session / Password Login - Demo` from the main collection first.
2. Open `American Latin Class - Python Analytics API`.
3. Run `Authentication Proof / Student Attendance Risk - No Token`; it must return `401`.
4. Run `Student Analytics`, `Branch Analytics` and `Teacher Analytics`; they inherit `Bearer {{session_token}}` and must return `200` after login.
5. Run logout from the main collection and repeat one protected analytics request; the revoked token must return `401`.

Notes:

- Do not commit real Google ID tokens, database passwords or session cookies.
- `login_email` and `login_password` are academic Postman verification credentials for the configured demo user.
- `google_id_token` is intentionally blank in the environment file.
- `session_token` is intentionally blank in the environment file and is filled by `Password Login - Demo`.
- The collection uses `{{base_url}}` for API calls and `{{site_url}}` for private page checks.
- The collection is configured with Bearer auth using `{{session_token}}`, while the browser flow still uses the `alc_session` cookie.
- Create/update requests save generated IDs back into the active Postman environment.
- Existing environment IDs point to records already loaded in AWS RDS; they can be used immediately for read/report tests.
- `analytics_base_url` is prepared for the Python FastAPI service through the frontend Nginx path `/api/analytics/v1`.

Automated evidence:

```
cd 06Code
npm run postman:evidence:local
```

This produces:

- `postman/evidence/postman-local-jwt-auth-evidence.md`
- `postman/evidence/postman-local-jwt-auth-evidence.json`